Share Programming Tips

C/C++ programming, Linux Programming, Android native application, Embedded systems

Saturday, July 14, 2018

Cross Compile C/C++ based programs and Run them on Android devices or smart phone

1. Introduction:

Learn how to cross compile pure C / C++ based native programs using different methods and execute it on your ARM based Android Smart-Phone's ADB terminal. By the end of this tutorial, you will be able to port any C / C++ based Native project in your Android smartphone.

2. Prerequisite Experience:

Before we get started,Let's make sure you are quite aware about following things.

1. You are experienced with C or C++ programming language.
2. You know how to compile any small C program using gcc
3. You are quite comfortable using the command line in Linux/Windows.
4. You are quite comfortable with Android Development tools like ADB
5. You understand the terms like "cross compilation", "Host machine"
6. You are quite comfortable with git and repo commands

3. When you need to cross compile C/C++ programs:

All Android's GUI based applications are JAVA based so if you want to process something in native code(C/C++) then you need to use JNI interface and call any c methods using JAVA interface and use it. But if you want to do something in pure native code or You want to port any existing programs/libraries available for Linux/Windows to Android smart-phones then you need to simply cross compile it.

Here we will shows you 3 different methods to cross compile this simple hello word program

```
================================ test.c ==============================
#include
int main()
{
 printf("Hello world \n");
 printf("It is cross compiled on Linux/Windows \n");
 printf("It is running on your Android Smart-Phone \n");
 return 0;
}

=====================================================================
```

Method 1: Static compilation using standalone toolchain

Here your program will cross compile against standard libc instead of Android's bionic library. Generated binary will have copy of all required functions. It will not use any library installed on android device so its binary size will be larger.

Supported Host OS: Linux only

Here you can download and install standalone toolchain by below command in Linux terminal
sudo apt-get install gcc-arm-linux-gnueabi

Now lets set some environment variable for cross compilation.
export CROSS_COMPILE=arm-linux-gnueabi-gcc

Now lets cross compile test.c file
arm-linux-gnueabi-gcc -static -o HelloWorldInAndroid test.c

HelloWorldInAndroid is the final binary compiled for ARM based Android device.

Method 2: Cross compilation using Android NDK's toolchain

Here your program will cross compile against bionic c library. All headers file are used from installed Android NDK.

Supported Host OS: Linux, Windows, Mac OS

First lets download and install Android NDK for your host OS machine from here
https://developer.android.com/tools/sdk/ndk/index.html

Android NDK compiles projects in particular patterns so lets perform below actions.
1) Create new folder named as "jni"
2) Copy your source file in jni folder
3) Prepare Android's special make file named as "Android.mk" in jni folder

```
jni/
 |
 + test.c
 + Android.mk
```

Copy below contains in Android.mk to build project

```
================================ Android.mk ============================
LOCAL_PATH := $(call my-dir)
# Lets clear other make file variable
include $(CLEAR_VARS)
# Give test.c as input file for cross compilation
LOCAL_SRC_FILES:= test.c
# Give name of the cross compiled binary
LOCAL_MODULE := HelloWorldInAndroid
# Give command to build executable
include $(BUILD_EXECUTABLE)
=====================================================================
```

Now lets cross compile the program

cd jni
/home//android-ndk-r7c/ndk-build

or

cd /home//android-ndk-r7c/
./ndk-build -C /home///jni

After compilation libs and obj folder will be created in parallel to jni or inside jni folder.

Generated binary will be at jni/libs/armeabi/HelloWorldInAndroid or
jni/../libs/armeabi/HelloWorldInAndroid

Method 3: Cross compilation using AOSP source code

Here your program will be cross compile against bionic c library. All headers file are used from AOSP android source code.

Supported Host OS: Linux, Windows, Mac OS

First lets download latest Android source code from AOSP project
https://source.android.com/source/downloading.html

Initializing a Build Environment
https://source.android.com/source/initializing.html

Now compile android source
cd Android-source
make

After compilation you will see some files in Android-source/out/target/product/generic
now download **agcc** python script in android source directory
cd Android-source
wget http://plausible.org/andy/agcc
chmod 777 ./agcc

Now lets set some environment variable for cross compilation.
export PATH=$PATH:./prebuilt/linux-x86/toolchain/arm-eabi-4.2.1/bin/

now cross compile your c program
agcc test.c -o HelloWorldInAndroid

HelloWorldInAndroid is the final binary compiled for ARM based Android device.

4. Run cross compiled binary in your android smart device.

Connect your Android Smart-Phone with USB cable to your Host machine
now install required driver and adb program if not installed

For windows find ADB driver from here
http://developer.android.com/tools/extras/oem-usb.html

For linux install it as below way
sudo add-apt-repository ppa:nilarimogard/webupd8
sudo apt-get update
sudo apt-get install android-tools-adb android-tools-fastboot

now open terminal and check you have installed correctly
sudo adb devices # it will list your connected device.

now copy your cross compiled binary in Android Smart-Phone's /data/local/atvc or /data/local/tmp folder
Because these folders have only executable permission for shell user
adb push /HelloWorldInAndroid /data/local/tmp

on Motorola's device
adb push /HelloWorldInAndroid /data/local/atvc
now run binary

adb shell # go in android shell
cd /data/local/tmp
./HelloWorldInAndroid

it will print below output
Hello world
It is cross compiled on Linux/Windows
It is running on your Android Smart-Phone

5. Conclusion

In this tutorial you learned how to corss compile your native C programs and port on your ARM based Android SmartPhone. I encourage you all to port any other open source native project to cross compile and port on your Android SmartPhone. All the best.

Posted by Jeegar Patel at 12:14 AM

2 comments:

1. *kkkblogger*November 7, 2018 at 3:12 PM

Hi,

Thanks for putting all methods in a place for ease of reading.

I have question regarding method 2. Is it possible to do the coding and use Visual Studio 2017 to compile AND DEBUG with Android-NDK?

I tried to create C++ console application for Android with VS 2017 but not successful.

ReplyDelete
Replies
Reply

2. *olegphorostyna*May 15, 2019 at 5:14 AM

Thank you! Great work!! But how to call that c code from Java while writing Android apps?

ReplyDelete
Replies
Reply

Add comment
Load more...

Newer Post Home

Subscribe to: Post Comments (Atom)

My Favourite post

Inline function vs Macros
https://stackoverflow.com/questions/1137575/inline-functions-vs-preprocessor-macros

Memory layout of C program
https://www.geeksforgeeks.org/memory-layout-of-c-program/

Linux kernel documentation
https://www.kernel.org/doc/html/latest/

Different kind of addressing in kernel
https://www.kernel.org/doc/Documentation/DMA-API-HOWTO.txt

Understand Page tabel here
https://www.geeksforgeeks.org/translation-lookaside-buffer-tlb-in-paging/

Device tree information:
https://events.static.linuxfound.org/sites/events/files/slides/petazzoni-device-tree-dummies.pdf

Convert hex to memory size
https://ss64.com/convert.html

Find me on StackOverFlow

Jeegar Patel
27k ●54 ●162 ●241

Followers

Awesome Inc. theme. Powered by Blogger.